

1) Choose a dataset

The dataset we chose comes from the Kaggle dataset “Edvard Munch Paintings Dataset” created by Kostiantyn Isaenkov. The dataset can be accessed at: [Edvard Munch Paintings](#). The dataset holds information about paintings created by the popular Norwegian artist Edvard Munch. This holds information such as titles to the artworks, years, dimensions, techniques, museum locations, image file references, and status. The CSV file used in the visualization is called “edvard_munch.csv.” There is a separate folder with the images, and all of those files are JPEGs. We were initially intrigued by the dataset due to its art-based information. We solidified our choice when we saw that it contained structured categorical and quantitative information that works well for interactive visualization and filtering in D3.js.

The main attributes used from the dataset include:

- title or name: the name of the painting
- year: the year the painting was created
- location: the museum or collection where the artwork is held
- technique: the artistic medium or technique used
- size or dimensions: the dimensions of the artwork
- status: whether the work is known, missing, or otherwise categorized
- filename or image URL fields: references to image files used in the timeline visualization

We had to use several data cleaning and preprocessing steps to make sure the dataset could be visualized correctly. The multiple cleaning processes are implemented directly inside the main.js file using D3.js functions and parsing functions. The first major issue we tackled was the inconsistent formatting in the year field. Some year entries contained a range instead of 1 number. To solve this problem, we created a parseYear() function that searches each year string for a 4-digit only value and converts it into an integer. If no valid year is written in the data, then the function returns null. This made sure that every painting was depicted on the timeline axis properly.

Our second issue with the dataset was the fact that the dimensions of the paintings were formatted incorrectly. The sizes of the paintings were stored as text strings with the numbers having commas instead of decimal points. We created a parseSizeToArea() function to create a standardized value by removing commas and periods before extracting the numerical width and height values. This function was then able to get the area of each painting by multiplying the two measurements together. This helped us with one of our filter selections. We were able to bucket the painting into three different size categories (“Small,” “Medium,” and “Large”). We used D3 quantiles to help place the paintings into different thresholds. With a helper function called buildSizeLabel(), each painting was then placed into one of the three categories based on which threshold it fell into.

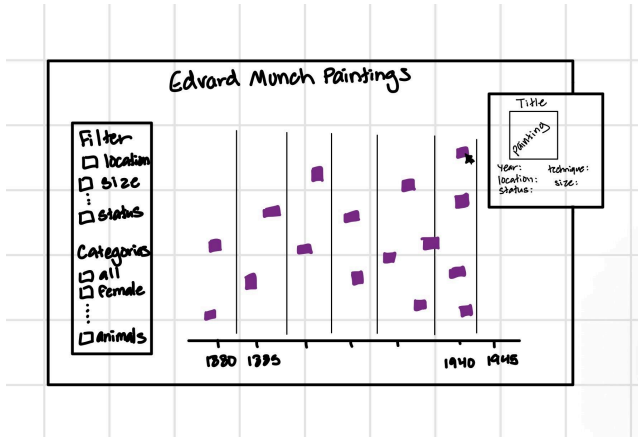
The dataset also contained missing or inconsistent text values for fields such as location, technique, and status. To make the interface cleaner and avoid blank filter options, a `normalizeValue()` function replaced missing or empty values with the default label “Unknown.” This ensured that all records remained usable in the visualization rather than being discarded. The decision to keep incomplete records rather than remove them preserved as much historical data as possible.

Our most difficult task in the dataset was dealing with image-related issues. We used the `resolveImageSource()` function to check for available image fields and to construct local image paths when needed. We added a preprocessing step after realizing the dataset was loading too slowly. To improve the loading speed of the images, we decided to use a `<picture>` element in the pop-up with AVIF and WebP sources before trying to use the original image file. These formats are much smaller than JPEGs, which helped the images load faster when updating the page and when loading the image in the pop-up box. The `main.js` file also creates alternate image paths automatically by removing the original `.jpeg` and tries `.avif` and `.webp` versions first. If those are missing, then the code retries the original image source and then displays an “Image file not found locally” message if needed.

Our visualization design required filtering logic for user interaction. The filter dropdown menus were generated from the cleaned dataset using the lists of sizes, locations, and techniques. The timeline updates whenever a filter selection is applied with the button, allowing users to explore different sections of the collection interactively.

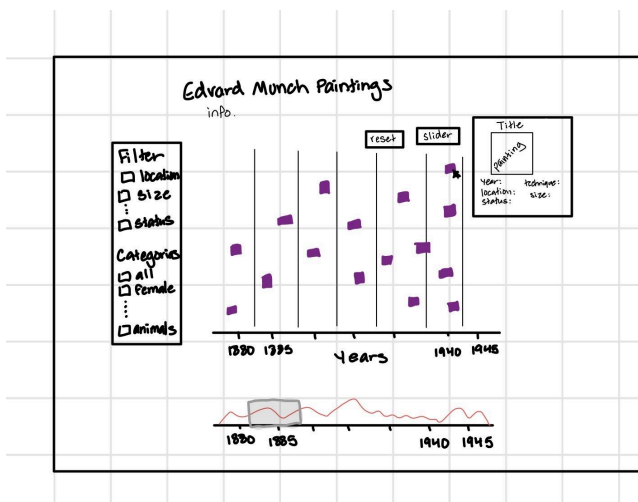
2) Paper prototypes (iterate)

Option 1:



This prototype is a timeline scatterplot where the years Munch produced paintings are encoded on the x-axis. Each item is encoded by a miniature version of the painting. When a painting is clicked on, a tooltip will pop up in the corner of the screen. The following information will be displayed: image of the painting, year created, location, status, technique, and size. On the left, there will be various filter options, including filter by year, size of painting, status, location, and category of the subject of the painting, which will affect how the painting is visualized. Since each item will be a downsized version of each painting, the background of the vis and any text will be in black and gray. This choice is to not overwhelm the user with competing colors. There will be smaller text under the title (Edvard Munch Paintings) with a short description of the vis as well as the interactivity that can be performed.

Option 1 iterative design:



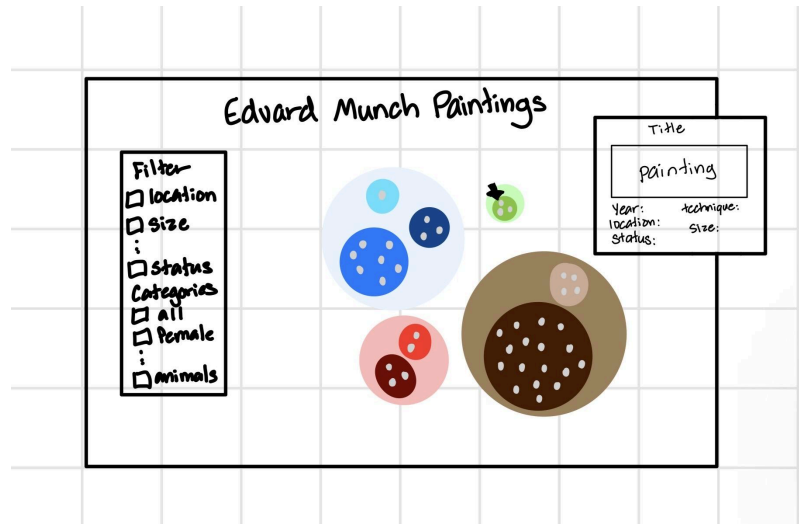
After choosing option one, we further developed the timeline scatterplot to show further details. This new sketch more clearly identifies where the information about the vis and interactivity will go with the text labeled “info,” under the title. We also decided to add a zoom-in feature. Under the timeline is an area chart of all the paintings over the years. There will be a slider that allows the user to zoom in on a subset of years. The width of the slider can also be changed up to the user's discretion. For example, they can make the width smaller for fewer years in the view or larger for more years. To switch between the standard all-year view and slider view will be two buttons at the top right relating to their respective modes.

ADD ANOTHER ITERATIVE DESIGN IF ANY CHANGES ARE MADE LIKE NO CATEGORIES OR A SEARCH FEATURE.

Option 1 iterative design 2:

After feedback from user testing and continued loading issues, we made some final pages to our design. As far as content, we clarified some of the instructions with bolded words, and added a hover feature over the “Reset year range” and “Slider” button to show how those work. We got rid of the status filter because it does not make sense to show lost paintings when there is no image for them. We also added accessibility features such as a screen reader to describe the vis and keyboard navigation. We also added color and a background image to make the overall vis more aesthetically pleasing. The biggest issue was our loading. Previously the images loaded to the screen extremely slowly but with good quality. This affected the responsiveness of our slider and user experience. After getting feedback from Prof. Mundy, we decided to create two image files, one for the thumbnails that was scaled down to 100x100 pixels because clear resolution was not necessary for the overall timeline. The other was the original image size only to be called for the popup when the painting is clicked on.

Option 2:



This prototype is a zoomable circle packing vis that is categorized by the mode color in each painting. The circles' hierarchy will depend on how many paintings have a mode of a single overall color. For example, among the 1800 paintings, if 1000 have a mode color of brown, the brown circle will be the largest. Within each overarching color, there will be subsets, such as light brown and dark brown, with dots representing each painting item. There will be a Zoom feature when you click on each circle. When a painting is clicked, a tooltip will appear in the corner of the screen. The following information will be displayed: the painting's image, year created, location, status, technique, and size. On the left, there will be various filter options, including year, painting size, status, location, and the subject category of the painting, that will affect how the painting is visualized. While we know that use of color can be difficult due to color blindness, our vis relies on color which is why the overarching circles are encoded with color. To be more accessible, we will not have certain colors right next to each other such as red and green. There will be smaller text under the title (Edvard Munch Paintings) with a short description of the vis as well as the interactivity that can be performed.

3) Task Analysis + Accessibility Analysis

Task Analysis

Task 1: Compare

Question: Which technique category contains more paintings: “Oil on canvas” or “Tempera on canvas”?

To complete this task, the user has to use the technique portion of the filter option. There, they will need to isolate each technique at a time. First, the user will select “Oil on canvas” from the technique dropdown, then press the apply button. The timeline updates to show only paintings that use that technique. The user will then visually estimate how many paintings (image marks/ image thumbnails) are in that category. When they're done, they will press the reset button to restore the timeline to its original state. The process would be repeated for “Tempura on canvas” and the counts compared. This task uses image thumbnails as marks, and the count of marks as the visual channel for comparing techniques. The filter interaction helps the user more easily differentiate the number of paintings in each category. If one filtered view shows a noticeable number of marks more than the other, the user can conclude that the specific category contains more paintings.

Task 2: Range

Question: What is the range of years represented in the Edvard Munch painting dataset?

To complete this task, the user has to look at the timeline’s x-axis. This would allow them to determine the earliest and latest years shown. The leftmost painting mark represents the earliest works, and the rightmost painting mark represents the latest works. The user can provide an approximate range by looking at the visual axis tick marks. The user could also click on paintings on the leftmost and rightmost sides to see the years they were created in the pop-up details. The range is the difference between the earliest and latest years. For example, if the earliest painting dates to around 1880 and the latest to around 1940, the range would be 60. This task uses image thumbnails as marks and horizontal position as the channel for the years.

Task 3: Order

Question: Order the selected paintings from earliest to latest by year.

To complete this task, there are two options. With both options, the user would have to use the search feature to look up the selected paintings. Then the user could look at the placement of the

painting thumbnails along the timeline. By scanning from left to right, the user can see the chronological order. The user could also disregard the placement on the x-axis and click on the painting to display the pop-up details, where the year information is stored. This method is recommended if two paintings are positioned very close together, so that the pop-up can give the exact year value. For example, if Painting A appears slightly to the left of Painting B, and Painting B appears to the left of Painting C, the user can conclude that the correct order is $A \rightarrow B \rightarrow C$ from earliest to latest. Each painting is represented by an image mark, and the horizontal position of the marks encodes the year the painting was made.

Accessibility Analysis

Task 1: Compare

To determine which category contains more paintings (“Oil on canvas” vs. “Tempera on canvas”), a non-visual user would not be able to rely on the use of visible marks. They would need text-based help. The non-visual marks would be converted into text entries, and the non-visual channel would be the explicit numerical count.

The user would need to follow these steps:

1. Navigate to the technique filter using a keyboard.
2. Select “Oil on canvas.”
3. Listen for a textual summary (i.e., “Showing 100 paintings”)
4. Repeat steps 1-3 for “Tempera on canvas.”
5. Compare the two values from the two techniques.

With this method, the user would gain a precise count of paintings within each technique, but they would lose the quick visual comparison of their distribution.

Task 2: Range

To determine the range of years, a user cannot look at the leftmost and rightmost positions. Instead, they rely on numerical values. The marks would be converted to text records, and the channel would encode numerical year values rather than horizontal position. Alternatively, the webpage could provide a summary that tells you when the range of paintings is.

The user would need to follow these steps:

1. Access a screen-reader-friendly list of paintings sorted by year.
2. Look for the smallest year value (earliest painting).
3. Look for the largest year value (latest painting).

4. Use subtraction to compute the range.

With this method, the user would gain an exact count for the years within the range, but they would lose the intuitive sense of how paintings are distributed over time.

Task 3: Order

To order selected paintings from earliest to latest based on year, a non-visual user would not be able to rely on spatial position. Instead, they would rely on numerical comparisons. The marks would be turned into individual text entries, and the channel would be encoding the numerical comparison of the years.

The user would need to follow these steps:

1. Search for selected paintings.
2. Use a screen reader to read the year value from the pop-up box.
3. Compare the years numerically.
4. Arrange the paintings from earliest to latest.

With this method, the user would gain precise ordering based on exact numbers, but would lose the ability to quickly scan the many items in the timeline.

4) Pilot the tasks and the (mostly working) vis

Here is the script and the steps taken for the pilot testing.

Speaker: "We are evaluating our visualization and are asking you, the participant, to complete some tasks using the visualization and then provide feedback about the visualization and experience. As a reminder, we are evaluating the visualization, not you as a participant, so you don't need to worry about being "right" as you complete these tasks. There are three tasks, followed by a brief feedback session. The whole pilot session should take under 5 minutes. Do you consent to participate?"

Participant: "Yes, I consent."

Speaker: "Thank you for agreeing to participate. We will start with the three tasks. Please 'think aloud' as you complete the task, meaning voice what you are thinking as you work through the task. Your first task is: Which technique category contains more paintings: "Oil on canvas" or "Tempera on canvas"?"

Participant: "Okay, do I just click anywhere on the screen to begin?"

Speaker: "Yes, that's fine, do whatever you want."

[Pause while participant completes task]

Speaker: "Okay, great job completing the first task. Your second task is: Finding the range of years represented in the Edvard Munch painting dataset.

[Pause while participant completes task]

Participant: "The range is 60 years."

Speaker: "Okay, thank you for completing that task. Your third and final task is: Order selected paintings from earliest to latest based on year. Here are the selected paintings: "Woman Standing in the Doorway, The Scream, and Cabaret

[Pause while participant completes task]

Participant: The order is Cabaret, The Scream, then Woman Standing in the Doorway.

Speaker: "That is the end of the third task. For this last bit, we welcome any feedback you may have about the visualization or about your process for completing the tasks. Thank you for helping with our project."

Participant: "I really like the idea of the visualization. If possible, maybe you could make the visualization a bit less laggy. Also, when I was exploring the years, the instructions regarding the slider feature were really unclear and confusing. The status feature of the filter section isn't really doing anything, so maybe you could remove that. One last thing is that you guys should consider making everything bigger on the webpage and overall a little more visually appealing."

Notes on how the participant completed the tasks

- Task 1
 - Opened the technique part of the filter section
 - Browsed quickly through all the options
 - "There's a lot of technique options."
 - Clicked on tempura first, counted, and nodded head
 - Hit reset
 - Clicked on oil next, counted, and nodded
 - Gave answer
- Task 2
 - Didn't interact with the page much
 - Moved the slider (had difficulty figuring that out)
 - Went back to the full screen of the timeline
 - Used the mouse to hover over the earliest and the latest year
- Task 3
 - Went to the search feature in the filter box
 - Typed in each painting name after asking for them again
 - Had a piece of paper, wrote down the order from earliest to latest
 - Found the year by reading the pop-up, not using the timeline

List of changes that we plan to make

Based on participants' feedback, we plan to make many changes to our visualization to improve functionality and usability. We will focus on performance optimization, clarity in instructions, refining some existing features, and overall visual aesthetics.

One main concern we share with the participant is that the webpage is very laggy due to the large dataset. To address this, we can reduce the size and resolution of thumbnail images to improve rendering speed. Next, we will display even clearer instructions so users won't feel confused when using any feature of the visualization. She noted that the slider was confusing, and we understand how that could happen. To fix this issue, we can redesign the slider to include clearer labels and instructions. Another suggestion we received was about the filter's status feature. We have decided to remove this attribute entirely since it doesn't provide meaningful variation in the dataset. Lastly, we will definitely change the webpage's overall appearance. It's very lackluster, and we didn't add any wow factor to the CSS stylesheet. So with that, we are going to make things larger, cleaner, and more engaging to the user.

5) Finalize your visualization

Olivia's webpage: [Final Project Visualization](#)

Rose's webpage: [Edvard Munch Painting Timeline](#)

This visualization was made with AI. Cursor was used to create boilerplate code and aid in the slider, Claude Code was used to create accessibility features, Codex was used to compress the file type and push to GitHub, ChatGPT was referenced for questions regarding styling.

5) User Testing

Person 1: Alissa Julien, double major in sociology and education studies

Speaker: "We are evaluating our visualization and are asking you, the participant, to complete some tasks using the visualization and then provide feedback about the visualization and experience. As a reminder, we are evaluating the visualization, not you as a participant, so you don't need to worry about being "right" as you complete these tasks. There are three tasks, followed by a brief feedback session. The whole pilot session should take under 5 minutes. Do you consent to participate?"

Participant: "Yes, I consent."

Speaker: "Thank you for agreeing to participate. We will start with the three tasks. Please 'think aloud' as you complete the task, meaning voice what you are thinking as you work through the task. Your first task is: Which technique category contains more paintings: "Oil on canvas" or "Tempera on canvas"?"

Participant: "Okay, so I'm figuring out if oil or tempera on canvas has the most paintings..."

[Pause while participant completes task]

Speaker: "Okay, great job completing the first task. Your second task is: Finding the range of years represented in the Edvard Munch painting dataset."

[Pause while participant completes task]

Participant: "The range is 1880 to like 1945."

Speaker: "Okay, thank you for completing that task. Your third and final task is: Order selected paintings from earliest to latest based on year. Here are the selected paintings: "Woman Standing in the Doorway, The Scream, and Cabaret"

[Pause while participant completes task]

Participant: The order is Cabaret, The Scream, then Woman Standing in the Doorway.

Speaker: "That is the end of the third task. For this last bit, we welcome any feedback you may have about the visualization or about your process for completing the tasks. Thank you for helping with our project."

Notes on how the participant completed the tasks

- Task 1
 - Opened the technique part of the filter section
 - Clicked canvas, oil: “that looks like a lot”
 - Looks at the data table but moves away as it does not provide necessary information
 - Clicked tempera, oil
 - Answers question
- Task 2
 - Resets filter
 - Looks at bottom of x-axis and evaluates
 - Answers question correctly
- Task 3
 - Went to the search feature in the filter box
 - Typed in each title
 - I clarified it was earliest to latest
 - Misspelled some cabaret
 - Correctly identified the order by looking at relative location on the timeline and not pop up

Summary: The user completed task 1 and 2 relatively fast. The average time was around 30 seconds which included finding the particular points listed in the task and analyzing. She had no issues navigating to the technique drop down or using the x-axis to determine the range of painting years. The user completed task 3 well by knowing to type in the title names by the search bar, but had some trouble with misspelling the names, so paintings would not pop up for a little bit. This task took around 2 minutes. Feedback: visually appealing, wish search bar had a spell check, would be cool if there was a number displayed to show how many canvas on oils or other techniques.

Person 2: Natalie Morris, double major in english and communications

Speaker: “We are evaluating our visualization and are asking you, the participant, to complete some tasks using the visualization and then provide feedback about the visualization and experience. As a reminder, we are evaluating the visualization, not you as a participant, so you don’t need to worry about being “right” as you complete these tasks. There are three tasks, followed by a

brief feedback session. The whole pilot session should take under 5 minutes. Do you consent to participate?"

Participant: "Yes, I consent."

Speaker: "Thank you for agreeing to participate. We will start with the three tasks. Please 'think aloud' as you complete the task, meaning voice what you are thinking as you work through the task. Your first task is: Which technique category contains more paintings: "Oil on canvas" or "Tempera on canvas"?"

[Pause while participant completes task]

Speaker: "Okay, great job completing the first task. Your second task is: Finding the range of years represented in the Edvard Munch painting dataset."

[Pause while participant completes task]

Participant: "The range is 1880s to 1943."

Speaker: "Okay, thank you for completing that task. Your third and final task is: Order selected paintings from earliest to latest based on year. Here are the selected paintings: "Woman Standing in the Doorway, The Scream, and Cabaret

[Pause while participant completes task]

Participant: The order is Cabaret, The Scream, then Woman Standing in the Doorway.

Speaker: "That is the end of the third task. For this last bit, we welcome any feedback you may have about the visualization or about your process for completing the tasks. Thank you for helping with our project."

Notes on how the participant completed the tasks

- Task 1
 - Goes to technique filters
 - Reasks the question
 - Clicks oil then tempera
 - Oil has a "bunch" in comparison
 - Answers question correctly
- Task 2
 - Resets to 'All' the painting
 - Looks at bottom of x-axis and evaluates
 - Answers question correctly

- Also looks to check the table for the exact states
- Task 3
 - Went to the search feature in the filter box
 - Typed in each title
 - Looks at popup and notes the year
 - Recognizes each title may have multiple paintings but picks earliest one
 - Answers question correctly

Summary: The user completed the task extremely well with task 1 and 2 taking around 20 seconds and task 3 around 35. The user had no issues navigating around the vis and applying/resetting any filters used. The user attempted to use the table of the dataset to look up the range values and , but realized it would be too difficult, so she opted to use what she gathered from the x-axis. This is with exception to scrolling all the way down to find the year '1943'. Feedback: easy to use, side tab/filters/keywords was helpful. It was clear that clicking on the painting would lead to more information.

Person 2: Ruby Donaldson, public health major, africana minor

Speaker: "We are evaluating our visualization and are asking you, the participant, to complete some tasks using the visualization and then provide feedback about the visualization and experience. As a reminder, we are evaluating the visualization, not you as a participant, so you don't need to worry about being "right" as you complete these tasks. There are three tasks, followed by a brief feedback session. The whole pilot session should take under 5 minutes. Do you consent to participate?"

Participant: "Yes, I consent."

Speaker: "Thank you for agreeing to participate. We will start with the three tasks. Please 'think aloud' as you complete the task, meaning voice what you are thinking as you work through the task. Your first task is: Which technique category contains more paintings: "Oil on canvas" or "Tempera on canvas"?"

[Pause while participant completes task]

Speaker: "Okay, great job completing the first task. Your second task is: Finding the range of years represented in the Edvard Munch painting dataset."

[Pause while participant completes task]

Participant: "The range is 1880 to 1940."

Speaker: "Okay, thank you for completing that task. Your third and final task is: Order selected paintings from earliest to latest based on year. Here are the selected paintings: "Woman Standing in the Doorway, The Scream, and Cabaret

[Pause while participant completes task]

Participant: The order is Cabaret, The Scream, then Woman Standing in the Doorway.

Speaker: "That is the end of the third task. For this last bit, we welcome any feedback you may have about the visualization or about your process for completing the tasks. Thank you for helping with our project."

Notes on how the participant completed the tasks

- Task 1
 - Repeats question
 - Filters by oil on canvas, then tempera on canvas
 - Visually determines oil has more
 - Answers question correctly
- Task 2
 - Scrolls down to x-axis
 - evaluates
 - Answers question correctly
- Task 3
 - Went to the search feature in the filter box
 - Searches 'woman' but then has to clarify 'woman standing in doorway'
 - Looks at popup and notes the year
 - Repeats for other two paintings
 - Recognizes each title may have multiple paintings but picks earliest one
 - Answers question correctly

Summary: The user completed the task extremely well with task 1 taking around 20 seconds, task 2 taking 10 seconds, and task 3 taking around 40 seconds. The user had no issue navigating around the vis and applying/resetting any filters used. It was clear that clicking on the painting would lead to more information. Feedback: enjoyed the paintings, liked the background of the website, easy to use.

7) Personal reflection

Rose: Out of all coding projects I've had in my 4 years here, this one had to be my absolute favorite. I really love how much freedom we had regarding visual design and the information displayed. Working on this project gave me practical skills for how much planning and iteration go into designing an effective data visualization. At the beginning of the project, Olivia and I were unsure which dataset to use and reviewed many options to find a good one. Thankfully, thanks to your suggestion, after narrowing our searches to art-related datasets, we were able to choose the current dataset we used for our project. We thought it would be the perfect type of information for a visually aesthetic webpage while still supporting analytical tasks like comparison, ordering, and identifying ranges.

My most challenging yet rewarding aspects of the project were preprocessing, compression, and using GitHub. Because our visualization used large image folders, we immediately ran into performance and storage issues. The original image folder contained 1800 images, which caused the webpage to take too long to load and caused problems when trying to upload the project to GitHub Pages. Between managing file sizes, playing around with smaller image formats, experimenting with compressed image formats, and troubleshooting GitHub push errors, it ended up taking far more time than we both expected, and it was extremely frustrating.

This process was rewarding because it was really nice to directly apply an elective class, Image Processing, to some of the code. While most of the techniques and methods did not fully work the way we needed it to, it was still valuable to experiment with various forms of image compression, lazy loading, and alternative image formats in a real-world visualization project.

Another frustrating part of the process was learning how to use GitHub more effectively. I used GitHub for the first time while I was abroad during my Junior year, and I haven't used it again until this semester. So, I had a hard time remembering how to handle repository management, merges, large-file issues, and deployment. Luckily, I was able to download an extension that helped me feel more comfortable using Git. That learning process was difficult, but it was also one of the skills I gained throughout the semester.

AI tools such as Cursor and ChatGPT (Codex) played a major role in the coding process. I primarily used AI to generate boilerplate code for us to build on. There are notes of what we kept in the comments. AI also helped a lot with debugging (giving suggestions as I'm actively coding), and I used it as a learning tool. For example, I used ChatGPT to help troubleshoot issues in the terminal, improve image-loading logic, explain concepts, and give examples of concepts I learned from the D3 website, etc. After a while, though, the use of AI started to have some major drawbacks. I felt that answers generated by ChatGPT were more complicated and clunkier than necessary. So, for me, AI works best for debugging existing code, and it saved me time in creating basic code structures.

Olivia:

This project was honestly really enjoyable for the most part as I got to pick an interesting topic and play around with it. The design process was not too bad in my opinion. We came up with two relatively good ideas and ultimately picked the timeline for ease. I did a lot of the brainstorming to how that would work, especially for when we were presenting our ideas to the class. I helped with little things in the beginning stages of the coding but mostly worked at the end. Throughout the beginning, I think the hardest part for me was not understanding why the images wouldn't load and the multiple different avenues rose and I talked about when it came to that. On my end, I was researching methods to help with the loading, used chatGPT to give me ideas, and also emailed back and forth with you (Dr. Williams) and Dr. Mundy. I did not actually code the loading issues in the beginning, but I had to figure out a lot of the thumbnail vs. original size with the already written code (the final method to get fast loading). Once I understood how to resize the images (using chatGPT to give me the commands for the terminal), the process for changing the calls in the code was not too difficult. I also ran into some issues with pushing everything to gitHub because the original images file was too large. Eventually, I was able to add the images in batches with minimal issues. Adding the slider function after the first iterative design was not too difficult because we did something similar in a lab, but I referenced Cursor to help with some of the styling, where that should go in the js code, and it generated some of the code, but any tweaks that were made was purely myself as there was many iterations of what the slider looked like. I also used Claude AI to help me create the accessibility features. For the styling of the vis, I referenced some ideas during class. I largely did that myself, but asked ChatGPT things such as which svg containers or what piece in the code dictates the size for the timeline box or what are some accessible fonts/colors that are accessible.